

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

1. IDENTIFICACIÓN DE LA GUÍA

Nombre de la guía:	Desarrollo de un microservicio web para gestión de reservas utilizando FastAPI y validación de datos con Pydantic
Código de la guía (No.):	001
Taller(es) o Laboratorio(s) aplicable(s):	Laboratorio Devops
Tiempo de trabajo práctico estimado:	2 horas
Asignatura(s) aplicable(s):	Aplicaciones y Servicios Web
Programa(s) Académico(s) / Facultad(es):	Tecnología en Desarrollo de software

COMPETENCIAS	CONTENIDO TEMÁTICO	INDICADOR DE LOGRO
Diseñar y desarrollar servicios web utilizando herramientas modernas de programación, aplicando estructuras de datos y validación de información para el intercambio de datos en formato JSON.	- Arquitectura básica de servicios web - Modelado de datos en Python - Validación de datos con Pydantic o dataclasses - Desarrollo de APIs con FastAPI - Uso de JSON para intercambio de datos - Control de versiones con Git y GitHub	El estudiante diseña modelos de datos estructurados, implementa validación de entradas mediante Pydantic o dataclasses y desarrolla endpoints funcionales en FastAPI que permiten recibir y devolver información en formato JSON.

2. FUNDAMENTO TEÓRICO

Los servicios web constituyen uno de los componentes fundamentales de las arquitecturas modernas de software, ya que permiten la comunicación entre diferentes aplicaciones mediante protocolos estándar, generalmente sobre HTTP. En este contexto, las **APIs REST** se han convertido en un enfoque ampliamente utilizado para la construcción de servicios web ligeros, escalables y fácilmente integrables con otros sistemas.

En el ecosistema de Python, **FastAPI** es un framework moderno diseñado para la construcción de APIs de alto rendimiento. FastAPI permite definir endpoints HTTP de manera clara y estructurada, facilitando la creación de servicios web robustos y bien documentados. Una de sus principales ventajas es la integración con **Pydantic**, una biblioteca que permite definir modelos de datos mediante clases de Python y validar automáticamente los datos recibidos en las solicitudes.

El modelado de datos es un proceso fundamental en el desarrollo de software, ya que permite representar entidades del mundo real mediante estructuras de programación. En Python, este modelado puede realizarse utilizando clases tradicionales, **dataclasses** o modelos de **Pydantic**, los

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

cuales ofrecen mecanismos para validar tipos de datos, rangos de valores, formatos específicos y restricciones estructurales.

En el desarrollo de servicios web, los datos suelen intercambiarse en formato **JSON (JavaScript Object Notation)**, un formato ligero de intercambio de datos que es fácilmente interpretable por múltiples lenguajes de programación. Por esta razón, es fundamental que los servicios web puedan recibir, validar y devolver datos correctamente estructurados.

Adicionalmente, el desarrollo colaborativo de software requiere herramientas de control de versiones que permitan registrar los cambios realizados por los diferentes integrantes de un equipo. En este sentido, **Git** y plataformas como **GitHub** permiten gestionar repositorios de código, realizar seguimiento de modificaciones y coordinar el trabajo entre múltiples desarrolladores.

En esta guía práctica, los estudiantes desarrollarán un microservicio web básico utilizando FastAPI, definirán modelos de datos mediante Pydantic o dataclasses, implementarán validación de información y utilizarán GitHub para gestionar el desarrollo colaborativo del proyecto.

3. OBJETIVOS

- Diseñar modelos de datos en Python que representen entidades de un sistema de reservas.
- Implementar validación de datos utilizando Pydantic o dataclasses.
- Desarrollar un microservicio web utilizando FastAPI que permita recibir y devolver datos en formato JSON.
- Implementar endpoints HTTP básicos (GET y POST).
- Utilizar Git y GitHub para gestionar el desarrollo colaborativo del proyecto.
- Comprender la estructura básica de un servicio web y su interacción mediante solicitudes HTTP.

4. RECURSOS REQUERIDOS

Para el desarrollo de esta práctica se requieren los siguientes recursos:

Equipos

- Computador personal o estación de trabajo por estudiante.

Herramientas de software

- Sistema operativo Linux o Windows.
- Python 3.10 o superior.
- FastAPI.
- Pydantic.
- Git.
- Cuenta en GitHub.
- Editor de código (por ejemplo: Visual Studio Code).

Material bibliográfico y recursos digitales

- Documentación oficial de FastAPI.
- Documentación oficial de Pydantic.

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

- Documentación oficial de Python.
- Guías básicas de uso de Git y GitHub.
- Repositorio de clase https://github.com/Juanmorales177809/apps_services.git

5. ASPECTOS DE SEGURIDAD

La práctica descrita en esta guía corresponde a una actividad de desarrollo de software, por lo cual no se identifican riesgos físicos o químicos asociados al uso de laboratorios experimentales.

Sin embargo, se recomienda tener en cuenta las siguientes consideraciones:

- Mantener una postura adecuada durante el uso prolongado del computador para evitar fatiga o lesiones musculares.
- Evitar la manipulación inadecuada de cables o conexiones eléctricas de los equipos.
- Realizar copias de seguridad periódicas del código desarrollado para evitar pérdida de información.

6. PROCEDIMIENTO O METODOLOGÍA PARA EL DESARROLLO

La práctica se desarrollará en equipos de trabajo conformados por dos a tres estudiantes.

Actividad 1: Creación del repositorio

1. Crear un repositorio en GitHub para el desarrollo del proyecto.
2. Clonar el repositorio en cada equipo de trabajo utilizando Git.
3. Crear un entorno virtual para el proyecto utilizando virtualenv o el módulo venv de Python.
4. Instalar las dependencias necesarias para el desarrollo del servicio web.
5. Generar un archivo **requirements.txt** que contenga las dependencias necesarias para ejecutar el servicio.
6. Incluir el archivo **.gitignore** que evite subir archivos innecesarios como el entorno virtual, temporales o carpetas generadas automáticamente.

Actividad 2: Definición de modelos de datos

A. Planteamiento del problema

La universidad desea desarrollar un sistema digital que permita registrar las reservas de salas utilizadas en actividades académicas.

Cada reserva debe almacenar información básica sobre:

- la sala que será utilizada,
- el usuario que realiza la reserva,
- la fecha y el horario en que se utilizará el espacio,
- el número de personas que asistirán,

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

- el estado de la reserva.

Para la primera versión del sistema se requiere construir un servicio web que permita registrar y consultar reservas utilizando solicitudes HTTP.

El servicio deberá recibir y devolver información en formato JSON.

En esta práctica no se utilizará una base de datos real. Las reservas se almacenarán temporalmente en memoria dentro del servidor.

B. Solución planteada

La solución consiste en desarrollar un microservicio utilizando FastAPI, el cual permita:

- recibir información de reservas mediante una solicitud POST
- consultar las reservas registradas mediante una solicitud GET

Para asegurar que la información recibida tenga la estructura correcta, se utilizará Pydantic, el cual permite definir modelos de datos en Python y validar automáticamente los datos recibidos en formato JSON.

C. Arquitectura de datos

El sistema utilizará una estructura de datos llamada Reserva, la cual representará la información que se registrará en el servicio web.

La estructura de la reserva contiene los siguientes campos:

- id_reserva
- id_sala
- id_usuario
- fecha
- hora_inicio
- hora_fin
- personas
- estado

Actividad 3: Desarrollo del microservicio

Con base en el ejemplo anterior, cada equipo deberá desarrollar su propio servicio web realizando las siguientes tareas:

1. Crear un archivo principal del proyecto llamado main.py.
2. Definir el modelo de datos **Reserva** utilizando Pydantic.
3. Implementar el endpoint **POST /reservas** que permita registrar una nueva reserva.

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

4. Implementar el endpoint **GET /reservas** que permita consultar todas las reservas registradas.
5. Almacenar las reservas utilizando una lista en memoria dentro del servidor.
6. Probar el funcionamiento del servicio utilizando la interfaz automática de FastAPI.

Actividad 4: Datos de prueba

El equipo deberá crear un archivo JSON con ejemplos de reservas para probar el funcionamiento del servicio.

El archivo debe incluir:

- Casos válidos.
- Casos con errores de formato o validación.

Actividad 5: Trabajo colaborativo

Cada integrante del equipo debe realizar al menos un commit desde su máquina local, indicando claramente el aporte realizado al proyecto.

Ejemplos de mensajes de commit:

- Aporte: modelos de datos con Pydantic.
- Aporte: implementación del endpoint POST /reservas.
- Aporte: implementación del endpoint GET /reservas.
- Aporte: creación de datos de prueba y documentación.

7. PARÁMETROS PARA ELABORACIÓN DEL INFORME

El equipo debe entregar:

- A. Enlace al repositorio del proyecto en GitHub que debe contener:
 - Código fuente del microservicio desarrollado.
 - Archivo JSON con datos de prueba.
 - Archivo README que incluya:
 - descripción del proyecto
 - instrucciones de instalación
 - instrucciones de ejecución
 - ejemplos de uso de los endpoints.

8. DISPOSICIÓN DE RESIDUOS

La actividad descrita en esta guía no genera residuos físicos o químicos. En consecuencia, no se requiere un procedimiento específico de disposición de residuos para esta práctica.

 Institución Universitaria Reacreditada en Alta Calidad	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

9. BIBLIOGRAFÍA

FastAPI. (2024). *FastAPI Documentation*. <https://fastapi.tiangolo.com>
Pydantic. (2024). *Pydantic Documentation*. <https://docs.pydantic.dev>
Python Software Foundation. (2024). *Python Documentation*. <https://docs.python.org>
Chacon, S., & Straub, B. (2014). *Pro Git*. Apress.
Fielding, R. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine.

Elaborado por:	Juan Carlos Morales Guerra
Revisado por:	Juan Carlos Morales Guerra
Versión:	001
Fecha:	27-02-2026